Tomatoma 백엔드 Spring Boot 학습 자료

> *완전 초보자를 위한 Tomatoma 백엔드 프로젝트 구조 및 Spring Boot 핵심 개념 학습 가이드*

목차

1. 프로젝트 전체 흐름
2. Spring Boot 핵심 개념
3. Spring AOP 이해 및 적용
4. 현재 코드 해석
5. REST API 엔드포인트 설계
6. 구현할 것 (TODO 체크리스트)

---

1. 프로젝트 전체 흐름

1.1 HTTP 요청의 여정: 계층별 흐름

사용자가 특정 음식의 위치를 찾고 싶을 때, HTTP 요청이 백엔드를 통과하는 과정입니다:

```
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 1. 클라이언트 (React Native / 웹 브라우저)                                                                    │
│    GET /api/places/1                                                                                         │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │ HTTP 요청
                ▼
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 2. Controller 계층 (FoodPlaceController)                                                                     │
│    - 요청 URL 분석                                                                                           │
│    - URL에서 파라미터 추출 (trendFoodId = 1)                                                                 │
│    - Service에 처리 위임                                                                                     │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │
                ▼
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 3. Service 계층 (PlacesService)                                                                              │
│    - 비즈니스 로직 실행                                                                                      │
│    - 데이터 검증                                                                                             │
│    - Repository에 쿼리 요청                                                                                  │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │
                ▼
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 4. Repository 계층 (FoodPlaceRepository)                                                                     │
│    - 데이터베이스 쿼리 실행                                                                                  │
│    - JPA가 자동으로 SQL 생성                                                                                 │
│    - 결과를 Entity 객체로 변환                                                                               │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │
                ▼
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 5. Database (PostgreSQL / H2)                                                                                │
│    - food_places 테이블에서 trend_food_id = 1인 모든 행 조회                                                 │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │ 데이터 반환
                ▼
┌─────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 4. Repository 계층 (역순)                                                                                    │
│    - List<FoodPlace> 반환                                                                                    │
└─────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │
                ▼
```

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 3. Service 계층 (역순)                                                                                        │
│    - Entity를 DTO로 변환 (보안/성능)                                                                          │
│    - 추가 비즈니스 로직 (정렬, 필터링 등)                                                                     │
│    - List<FoodPlaceDTO> 반환                                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │
                ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 2. Controller 계층 (역순)                                                                                     │
│    - ResponseDTO에 담기                                                                                       │
│    - HTTP 상태 코드 설정 (200 OK)                                                                             │
│    - JSON 직렬화 (@RestController가 자동 처리)                                                                │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                │ JSON 응답
                ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ 1. 클라이언트 (React Native / 웹 브라우저)                                                                    │
│    응답 JSON:                                                                                                 │
│    {                                                                                                          │
│      "status": "success",                                                                                     │
│      "data": [                                                                                                │
│        {                                                                                                      │
│          "id": 1,                                                                                             │
│          "name": "명동 계란밥",                                                                               │
│          "address": "서울시 중구 명동 111",                                                                   │
│          "rating": 4.5                                                                                        │
│        },                                                                                                     │
│        ...                                                                                                    │
│      ]                                                                                                        │
│    }                                                                                                          │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

1.2 각 계층의 역할

Controller (컨트롤러) - 요청의 진입점

- HTTP 요청을 받음
- 요청에서 파라미터/본문 추출
- Service에 처리 요청
- 응답을 JSON으로 변환해 클라이언트에 전송
- 예: GET `/api/places/1` 요청 받으면 foodPlaceId=1 추출

Service (서비스) - 비즈니스 로직의 핵심

- Controller에서 받은 데이터 검증
- Repository를 사용해 데이터 조회/생성/수정
- 데이터 변환 (Entity → DTO)
- 복잡한 비즈니스 로직 처리
- 거의 모든 비즈니스 로직이 여기에 위치

Repository (저장소) - 데이터베이스 접근

- Entity와 데이터베이스 테이블의 "다리"
- JPA가 `findByName()` 같은 메서드를 자동 구현
- 쿼리 결과를 Entity 객체로 자동 변환

Entity (엔티티) - 데이터베이스 테이블의 표현

- 데이터베이스의 한 행(row)을 나타내는 Java 클래스
- `@Entity`, `@Column` 같은 어노테이션으로 데이터베이스와 매핑
- 테이블의 구조를 반영

DTO (Data Transfer Object) - 통신용 객체

- Entity를 그대로 보내지 않고 필요한 필드만 추려서 보냄
- 보안: 내부 로직이 보이지 않음

- 성능: 불필요한 필드 전송 안 함
- 예: FoodPlace Entity의 모든 필드를 보내지 않고, 클라이언트가 필요한 필드만 FoodPlaceDTO로 보냄

---

2. Spring Boot 핵심 개념

2.1 @SpringBootApplication

역할: Spring Boot 애플리케이션의 시작점

```
// TomatomaApplication.java
@SpringBootApplication
@EnableScheduling
public class TomatomaApplication {
    public static void main(String[] args) {
        SpringApplication.run(TomatomaApplication.class, args);
    }
}
```

@SpringBootApplication이 하는 일:

1. **@Configuration**: Spring이 설정 클래스임을 인식 (설정 파일 처럼 취급)
2. **@ComponentScan**: 이 클래스가 있는 패키지와 그 하위 패키지의 모든 `@Component`, `@Controller`, `@Service`, `@Repository` 클래스를 찾아 자동 등록
3. **@EnableAutoConfiguration**: Spring Boot가 클래스패스(라이브러리)를 보고 필요한 것들을 자동으로 설정 (예: PostgreSQL 드라이버가 있으면 자동으로 DataSource 설정)

쉽게 말해:

- Spring Boot 앱이 시작될 때 이 클래스를 찾음
- 이 패키지 아래의 모든 Spring 빈들을 등록
- 필요한 자동 설정 적용

2.2 @RestController vs @Controller

@RestController (우리 프로젝트에서 사용)

```
@RestController
@RequestMapping("/api/trends")
public class TrendFoodController {

    @GetMapping
    public ResponseEntity<ResponseDTO> getAllTrends() {
        // 반환 값이 자동으로 JSON으로 변환됨
        return ResponseEntity.ok(new ResponseDTO("success", trends));
    }
}
```

특징:

- `@Controller + @ResponseBody` 합친 것
- 메서드의 반환값이 **자동으로 JSON으로 직렬화**됨
- REST API 개발용

@Controller

```
@Controller
@RequestMapping("/views")
public class PageController {

    @GetMapping("/home")
    public String getHomePage(Model model) {
        // 문자열을 반환하면 HTML 템플릿 이름으로 취급
```

```
        // resources/templates/home.html 찾아서 렌더링
        return "home";
    }
}
```

특징:

- 전통적인 웹 애플리케이션용 (HTML 페이지 반환)
- 메서드의 반환값이 템플릿 이름(예: "home")으로 취급
- Model 객체에 데이터 담아 템플릿에 전달

정리:

- REST API 만들려면: **@RestController**
- HTML 페이지 렌더링하려면: **@Controller**

2.3 @Service, @Repository, @Component - IoC와 DI

먼저 알아야 할 개념: IoC (Inversion of Control, 제어의 역전)

IoC 없을 때 (수동 관리):

```
public class PlacesService {
    // 직접 객체 생성
    private FoodPlaceRepository repository = new FoodPlaceRepository();

    public List<FoodPlace> findPlaces(Long trendFoodId) {
        return repository.findByTrendFoodId(trendFoodId);
    }
}
```

문제점:

- Service가 Repository 구현체와 강하게 결합
- Repository를 다른 구현으로 바꾸려면 Service 코드 수정 필요
- 테스트할 때 가짜(Mock) Repository로 쉽게 교체 불가

IoC 적용 후 (Spring이 관리):

```
@Service
public class PlacesService {
    // Spring이 생성 및 주입해줌
    private FoodPlaceRepository repository;

    // 생성자 주입 (생성자가 있으면 Spring이 자동으로 주입)
    public PlacesService(FoodPlaceRepository repository) {
        this.repository = repository;
    }

    public List<FoodPlace> findPlaces(Long trendFoodId) {
        return repository.findByTrendFoodId(trendFoodId);
    }
}
```

장점:

- Service는 Repository 인터페이스에만 의존 (느슨한 결합)
- 다른 구현으로 쉽게 교체 가능
- 테스트할 때 Mock 객체로 교체 가능

DI (Dependency Injection, 의존성 주입)

의존성: 어떤 객체가 다른 객체를 필요로 하는 관계

주입: Spring이 자동으로 필요한 객체를 생성해서 넘겨주는 것

@Service, @Repository, @Component 차이

```java
// 1. 비즈니스 로직 처리 계층
@Service
public class PlacesService {
    // Repository 주입
    private final FoodPlaceRepository repository;

    public PlacesService(FoodPlaceRepository repository) {
        this.repository = repository;
    }
}

// 2. 데이터베이스 접근 계층
@Repository
public interface FoodPlaceRepository extends JpaRepository<FoodPlace, Long> {
    List<FoodPlace> findByTrendFoodId(Long trendFoodId);
}

// 3. 특정 역할이 없는 일반 컴포넌트
@Component
public class LoggingUtils {
    public void log(String message) {
        System.out.println(message);
    }
}
```

역할의 차이:

- `@Service` : 비즈니스 로직을 처리하는 객체임을 명시 (아무 기능은 없지만 가독성 향상)
- `@Repository` : 데이터베이스 접근 객체임을 명시 + 데이터베이스 예외를 Spring 예외로 변환
- `@Component` : 그냥 Spring이 관리하는 일반 빈

사실:

- 모두 `@Component` 의 특수화일 뿐이므로 기술적으로는 `@Component` 로 다 가능
- 하지만 명확한 의도 표현을 위해 구분해서 사용

2.4 @Autowired vs 생성자 주입

@Autowired (필드 주입) - 옛 방식

```java
@Service
public class PlacesService {
    // Spring이 자동으로 FoodPlaceRepository 타입의 빈을 찾아 주입
    @Autowired
    private FoodPlaceRepository repository;

    public List<FoodPlace> findPlaces(Long id) {
        return repository.findByTrendFoodId(id);
    }
}
```

문제점:

- 주입되지 않으면 null이 되는데, 코드 실행 시간에 NullPointerException 발생
- 테스트할 때 필드를 임의로 설정하기 어려움
- 순환 의존성(A→B→A) 문제 감지 어려움

생성자 주입 (Constructor Injection) - 권장 방식

```java
@Service
public class PlacesService {
    // final로 선언해 불변성 보장 (생성 후 변경 불가)
    private final FoodPlaceRepository repository;

    // 생성자에서 의존성 받음
    // Spring이 자동으로 FoodPlaceRepository를 찾아서 주입
    public PlacesService(FoodPlaceRepository repository) {
        this.repository = repository;
    }

    public List<FoodPlace> findPlaces(Long id) {
        return repository.findByTrendFoodId(id);
    }
}
```

장점:

- 필드가 final이므로 변경 불가 (안전성)
- 생성자에서 검증 가능 (null 체크 등)
- 테스트할 때 쉽게 Mock 객체 주입 가능
- 순환 의존성을 생성자 호출 시점에 감지 (앱이 시작 안 됨)

정리:

```
필드 주입 (@Autowired) → 생성자 주입로 변경 권장
```

2.5 @Entity, @Id, @Column - JPA ORM

ORM이란?

Object-Relational Mapping: 자바 객체를 데이터베이스 테이블 행과 매핑하는 기술

매핑 없이:

```java
-- SQL 직접 작성
SELECT id, name, category FROM categories WHERE id = 1;
-- 결과를 직접 객체로 변환
Category category = new Category();
category.setId(rs.getLong("id"));
category.setName(rs.getString("name"));
...
```

JPA ORM 사용:

```java
// 자동으로 위 과정이 모두 처리됨
Category category = categoryRepository.findById(1L).orElse(null);
```

@Entity - 데이터베이스 테이블과 매핑

```java
@Entity  // 이 클래스가 데이터베이스의 한 테이블임을 표시
@Table(name = "categories")  // 테이블명 명시 (없으면 클래스명 소문자)
public class Category {
    // ...
}
```

역할:

- 이 클래스의 각 필드가 테이블의 컬럼과 매핑됨
- 각 객체 인스턴스가 테이블의 한 행(row)

@Id - 주 키(Primary Key)

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
```

의미:

- `@Id` : 이 필드가 테이블의 주 키(기본 키)임을 표시
- `@GeneratedValue(GenerationType.IDENTITY)` : 새 행 추가할 때 id를 자동으로 증가 (1, 2, 3, ...)

선택지:

```
// 1. IDENTITY: 데이터베이스가 값 생성 (MySQL, PostgreSQL, H2)
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

// 2. SEQUENCE: 데이터베이스 시퀀스에서 값 생성 (Oracle)
@GeneratedValue(strategy = GenerationType.SEQUENCE)
private Long id;

// 3. AUTO: 방언에 따라 자동 선택
@GeneratedValue(strategy = GenerationType.AUTO)
private Long id;

// 4. 수동: 개발자가 직접 설정
private Long id;
```

@Column - 컬럼 속성 정의

```
@Column(nullable = false, unique = true, length = 50)
private String name;

@Column(length = 7)  // Hex color code
private String color;
```

속성:

- `nullable = false` : NULL 값 허용 안 함 (not null 제약)
- `unique = true` : 중복 값 허용 안 함 (유니크 제약)
- `length = 50` : 문자열 최대 길이 50
- `updatable = false` : 생성 후 수정 불가 (created_at 같은 필드)

실제 예시: Category 클래스와 categories 테이블 매핑

```
@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @Column(length = 7)
    private String color;

    @Column(length = 10)
```

```
    private String icon_emoji;
}
```

생성되는 SQL:

```
CREATE TABLE categories (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,
    color VARCHAR(7),
    icon_emoji VARCHAR(10)
);
```

2.6 JpaRepository와 쿼리 메서드

JpaRepository 상속

```
@Repository
public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByName(String name);
}
```

제네릭 파라미터:

- `<Category, Long>` : 이 Repository가 관리하는 Entity는 Category, 주 키 타입은 Long
- Category Entity에서 `@Id` 가 Long 타입이므로 Long 사용

JpaRepository가 제공하는 기본 메서드

```
// 모두 자동으로 구현됨
categoryRepository.findAll();              // 모든 행 조회
categoryRepository.findById(1L);           // 주 키로 조회 (Optional 반환)
categoryRepository.save(category);          // 저장 (없으면 insert, 있으면 update)
categoryRepository.delete(category);        // 삭제
categoryRepository.deleteById(1L);         // 주 키로 삭제
categoryRepository.count();                // 행의 개수
categoryRepository.existsById(1L);         // 존재 여부 확인
```

쿼리 메서드 - 자동 SQL 생성

개발자가 메서드 이름으로 쿼리를 "선언"하면, Spring Data JPA가 자동으로 SQL을 생성합니다.

규칙: `findBy[필드명][조건](파라미터)`

```
@Repository
public interface CategoryRepository extends JpaRepository<Category, Long> {

    // 1. findByName - name이 정확히 일치하는 행 조회
    Optional<Category> findByName(String name);
    // 자동 생성 SQL: SELECT * FROM categories WHERE name = ?

    // 아직 구현되지 않음, 예시용:

    // 2. findByNameContaining - name이 포함하는 모든 행 조회
    // List<Category> findByNameContaining(String keyword);
    // 자동 생성 SQL: SELECT * FROM categories WHERE name LIKE ?

    // 3. findByColorAndId - color와 id 모두 일치하는 행 조회
    // Optional<Category> findByColorAndId(String color, Long id);
    // 자동 생성 SQL: SELECT * FROM categories WHERE color = ? AND id = ?

    // 4. findAllOrderByName - 모든 행을 name으로 정렬해서 조회
```

```
    // List<Category> findAllOrderByName();
    // 자동 생성 SQL: SELECT * FROM categories ORDER BY name
}
```

Spring Data JPA가 인식하는 키워드:

- `find...By` : 조회
- `count...By` : 개수 세기
- `delete...By` : 삭제
- `exists...By` : 존재 여부
- `And` , `Or` : 조건 결합
- `GreaterThan` , `LessThan` : 크기 비교 (>, <)
- `Between` : 범위 (예: searchFrequencyBetween(100, 1000))
- `Containing` , `Like` : 문자열 포함
- `OrderBy...Asc` , `OrderBy...Desc` : 정렬

효과:

- 개발자는 메서드만 선언 (구현 안 함)
- Spring Data JPA가 런타임에 자동으로 구현체 생성
- SQL 코드 작성 없음

2.7 @Scheduled - 정기적 작업 자동 실행

현재 프로젝트에서는 `@EnableScheduling` 이 TomatomaApplication에 있고, `application.yml` 에 스케줄 설정이 있습니다.

```
@SpringBootApplication
@EnableScheduling  // 스케줄 기능 활성화
public class TomatomaApplication {
    public static void main(String[] args) {
        SpringApplication.run(TomatomaApplication.class, args);
    }
}
```

```
# application.yml
scheduler:
  trend:
    cron: "0 0 * * * ?"  # 매시간 정각에 실행
  enabled: true
```

@Scheduled 예시 (구현할 것)

```
@Service
public class TrendUpdaterScheduler {

    private final TrendService trendService;

    public TrendUpdaterScheduler(TrendService trendService) {
        this.trendService = trendService;
    }

    // 매시간 정각에 자동 실행 (크론 표현식)
    @Scheduled(cron = "0 0 * * * ?")
    public void updateTrendFoods() {
        System.out.println("트렌드 데이터 업데이트 시작: " + LocalDateTime.now());

        try {
            // 외부 API에서 최신 트렌드 음식 조회
            List<TrendFood> latestTrends = trendService.fetchLatestTrends();

            // 데이터베이스에 저장
```

```
            trendService.saveAllTrends(latestTrends);

            System.out.println("트렌드 데이터 업데이트 완료");
        } catch (Exception e) {
            System.err.println("트렌드 업데이트 중 에러: " + e.getMessage());
        }
    }

    // 5초마다 실행
    @Scheduled(fixedRate = 5000)
    public void logMemoryUsage() {
        long usedMemory = Runtime.getRuntime().totalMemory() -
                          Runtime.getRuntime().freeMemory();
        System.out.println("현재 메모리 사용량: " + usedMemory / 1024 / 1024 + "MB");
    }
}
```

@Scheduled 옵션:

```
// 1. Cron 표현식 (시간 관련 정확한 제어)
@Scheduled(cron = "0 0 * * * ?")  // 매시간 정각
// Cron 형식: 초(0-59) 분(0-59) 시(0-23) 일(1-31) 월(1-12) 요일(0-6)
// ? = 지정하지 않음 (보통 요일이나 일에 사용)

@Scheduled(cron = "0 0 9 * * MON")  // 월요일 09:00

// 2. fixedRate: 마지막 시작부터 밀리초 단위 간격
@Scheduled(fixedRate = 60000)  // 1분마다

// 3. fixedDelay: 마지막 종료부터 밀리초 단위 간격
@Scheduled(fixedDelay = 60000)  // 실행 후 1분 대기 후 다시 실행

// 4. initialDelay: 첫 실행 전 대기 시간
@Scheduled(fixedRate = 60000, initialDelay = 10000)  // 10초 후 시작, 1분마다
```

2.8 @GetMapping, @PostMapping - REST API 매핑

HTTP 메서드와 URL 경로를 메서드에 매핑합니다.

```
@RestController
@RequestMapping("/api/trends")  // 기본 경로
public class TrendFoodController {

    // GET /api/trends - 모든 트렌드 조회
    @GetMapping
    public ResponseEntity<ResponseDTO> getAllTrends() {
        List<TrendFood> trends = trendService.getAllTrends();
        return ResponseEntity.ok(
            new ResponseDTO("success", trends, null)
        );
    }

    // GET /api/trends/1 - 특정 트렌드 조회
    @GetMapping("/{id}")
    public ResponseEntity<ResponseDTO> getTrendById(@PathVariable Long id) {
        TrendFood trend = trendService.getTrendById(id);
        return ResponseEntity.ok(
            new ResponseDTO("success", trend, null)
        );
    }
```

```java
    // GET /api/trends/search?q=버터 - 검색
    @GetMapping("/search")
    public ResponseEntity<ResponseDTO> searchTrends(
        @RequestParam String q
    ) {
        List<TrendFood> results = trendService.searchByName(q);
        return ResponseEntity.ok(
            new ResponseDTO("success", results, null)
        );
    }

    // POST /api/trends - 새 트렌드 추가
    @PostMapping
    public ResponseEntity<ResponseDTO> createTrend(
        @RequestBody TrendFoodDTO dto
    ) {
        TrendFood saved = trendService.saveTrend(dto);
        return ResponseEntity.status(201).body(
            new ResponseDTO("success", saved, null)
        );
    }
}
```

어노테이션 상세 설명

@RequestMapping: 클래스 레벨에서 기본 경로 설정

```java
@RestController
@RequestMapping("/api/trends")  // 모든 경로가 /api/trends로 시작
public class TrendFoodController { ... }
```

@GetMapping, **@PostMapping**, **@PutMapping**, **@DeleteMapping**: HTTP 메서드 매핑

```java
@GetMapping              // GET /api/trends
@PostMapping            // POST /api/trends
@GetMapping("/{id}")    // GET /api/trends/{id}
@PutMapping("/{id}")    // PUT /api/trends/{id}
@DeleteMapping("/{id}") // DELETE /api/trends/{id}
```

@PathVariable: URL 경로의 변수 추출

```java
@GetMapping("/{trendId}/places/{placeId}")
public void example(
    @PathVariable Long trendId,    // /123/places/456 에서 123
    @PathVariable Long placeId     // 456
) { ... }
```

@RequestParam: URL 쿼리 파라미터 추출

```java
@GetMapping("/search")
public void example(
    @RequestParam String q,        // /search?q=버터 에서 "버터"
    @RequestParam(defaultValue = "1") Integer page  // 기본값 설정 가능
) { ... }
```

@RequestBody: HTTP 요청 본문(body)을 객체로 변환

```java
@PostMapping
public void createTrend(
    @RequestBody TrendFoodDTO dto  // JSON을 TrendFoodDTO로 자동 변환
```

```
) {
    // dto.getName() 등으로 접근
}

// 클라이언트가 보낸 JSON:
// {
//   "name": "버터 계란 밥",
//   "category": "한식",
//   "searchFrequency": 8500
// }
```

2.9 application.yml - 설정 파일

현재 프로젝트의 `application.yml` 해석:

```yaml
spring:
  application:
    name: tomatoma-backend  # 애플리케이션 이름

  # 데이터베이스 설정
  datasource:
    url: jdbc:h2:mem:tomatoma      # H2 인메모리 데이터베이스 (개발용)
    driver-class-name: org.h2.Driver
    username: sa                    # 기본 사용자명
    password:                       # 비밀번호 없음

  h2:
    console:
      enabled: true                 # H2 웹 콘솔 활성화 (http://localhost:8080/h2-console)
      path: /h2-console

  # JPA/Hibernate 설정
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect  # H2용 SQL 방언
    hibernate:
      ddl-auto: create-drop  # 애플리케이션 시작할 때 테이블 생성, 종료할 때 삭제
                             # 개발할 때만 사용 (운영: validate 또는 none)
    defer-datasource-initialization: true  # data.sql 로드 전 테이블 생성 기다리기
    show-sql: false                        # SQL 출력 여부
    properties:
      hibernate:
        format_sql: true      # 출력되는 SQL 예쁘게 포맷
        use_sql_comments: true  # SQL에 주석 추가

  # CORS 설정 (React Native/웹에서 접근 허용)
  web:
    cors:
      allowed-origins: "http://localhost:3000,http://localhost:5173"
      allowed-methods: GET,POST,PUT,DELETE,OPTIONS
      allowed-headers: "*"
      allow-credentials: true

# 서버 설정
server:
  port: 8080                    # 포트 번호
  servlet:
    context-path: /api          # 모든 엔드포인트가 /api로 시작

# 외부 API 설정
google:
  maps:
    api-key: ${GOOGLE_MAPS_API_KEY:your_google_maps_api_key}
```

```yaml
    # ${변수명:기본값} 형식
    # 환경변수에서 GOOGLE_MAPS_API_KEY를 찾고, 없으면 기본값 사용

external-apis:
  serpapi:
    api-key: ${SERPAPI_API_KEY:placeholder_serpapi_key}
    base-url: https://serpapi.com

# 스케줄러 설정
scheduler:
  trend:
    cron: "0 0 * * * ?"  # 매시간
  enabled: true

# 로깅 설정
logging:
  level:
    root: INFO                        # 기본 레벨
    com.tomatoma: DEBUG              # 우리 패키지는 DEBUG로
    org.springframework.web: INFO
    org.hibernate: WARN              # Hibernate는 경고만
  pattern:
    console: "%d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
```

key 설정 활용:

```java
// 설정 값을 Java 코드에서 사용
@Component
public class GoogleMapsConfig {

    @Value("${google.maps.api-key}")  // application.yml에서 값 읽기
    private String apiKey;

    public String getApiKey() {
        return apiKey;
    }
}
```

---

3. Spring AOP 이해 및 적용

3.1 현재 상태

이 프로젝트에는 Spring AOP가 아직 적용되지 않았습니다.

확인 사항:

- `TomatomaApplication.java` 에 `@EnableAspectJAutoProxy` 없음
- `pom.xml` 에 `spring-boot-starter-aop` 의존성 없음
- Aspect 클래스 파일 없음

3.2 AOP란 무엇인가?

AOP (Aspect-Oriented Programming, 관점 지향 프로그래밍)

횡단 관심사(Cross-Cutting Concerns)를 분리하는 프로그래밍 방식입니다.

횡단 관심사란?

```
비즈니스 로직:
- 트렌드 음식 조회
- 음식점 검색
- 카테고리 관리
```

```
횡단 관심사 (여러 메서드에서 반복되는 것들):
- 로깅 (어떤 메서드가 몇 초 걸렸는가)
- 보안 (사용자 인증 확인)
- 트랜잭션 (DB 저장 실패 시 롤백)
- 예외 처리 (에러 로그 및 응답)
```

AOP 없을 때:

```java
@Service
public class TrendService {

    public List<TrendFood> getAllTrends() {
        long startTime = System.currentTimeMillis();
        System.out.println("getAllTrends 메서드 시작");

        try {
            // 실제 비즈니스 로직
            List<TrendFood> trends = trendFoodRepository.findAll();

            long duration = System.currentTimeMillis() - startTime;
            System.out.println("getAllTrends 실행 시간: " + duration + "ms");
            return trends;

        } catch (Exception e) {
            System.err.println("getAllTrends 에러: " + e.getMessage());
            throw e;
        }
    }

    public TrendFood getTrendById(Long id) {
        long startTime = System.currentTimeMillis();
        System.out.println("getTrendById 메서드 시작");

        try {
            // 실제 비즈니스 로직
            TrendFood trend = trendFoodRepository.findById(id).orElse(null);

            long duration = System.currentTimeMillis() - startTime;
            System.out.println("getTrendById 실행 시간: " + duration + "ms");
            return trend;

        } catch (Exception e) {
            System.err.println("getTrendById 에러: " + e.getMessage());
            throw e;
        }
    }

    // 모든 메서드에서 로깅과 예외 처리가 반복됨!
}
```

문제점:

- 로깅 코드가 비즈니스 로직을 오염시킴
- 모든 메서드에서 반복됨
- 유지보수 어려움

AOP 적용 후:

```java
@Service
public class TrendService {
```

```java
    // 비즈니스 로직에만 집중
    public List<TrendFood> getAllTrends() {
        return trendFoodRepository.findAll();
    }

    public TrendFood getTrendById(Long id) {
        return trendFoodRepository.findById(id).orElse(null);
    }
}

// 로깅은 별도의 Aspect에서 처리
@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* com.tomatoma.service.*.*(..))") // Service 계층의 모든 메서드
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();
        String methodName = joinPoint.getSignature().getName();

        System.out.println("[START] " + methodName);

        try {
            Object result = joinPoint.proceed();  // 실제 메서드 실행

            long duration = System.currentTimeMillis() - startTime;
            System.out.println("[END] " + methodName + " - 실행 시간: " + duration + "ms");

            return result;
        } catch (Exception e) {
            System.err.println("[ERROR] " + methodName + " - 에러: " + e.getMessage());
            throw e;
        }
    }
}
```

장점:

- 비즈니스 로직이 깔끔함
- 로깅, 보안 등이 한 곳에서 관리됨
- 변경이 쉬움

3.3 AOP 핵심 개념

1. Aspect (관점)

```
로깅, 보안, 트랜잭션 등 횡단 관심사를 처리하는 모듈
```

2. Pointcut (포인트컷)

```
어느 메서드에 Aspect를 적용할 것인지 지정하는 표현식
```

```java
// Service 패키지의 모든 메서드에 적용
@Around("execution(* com.tomatoma.service.*.*(..))")

// TrendService 클래스의 모든 메서드에 적용
@Around("execution(* com.tomatoma.service.TrendService.*(..))")

// "get"으로 시작하는 메서드에 적용
@Around("execution(* com.tomatoma..*.get*(..))")
```

```
// @Service 어노테이션이 있는 클래스의 모든 메서드에 적용
@Around("@within(org.springframework.stereotype.Service)")
```

3. Advice (어드바이스)

```
실제 실행되는 코드
```

```
@Aspect
@Component
public class MyAspect {

    // Before: 메서드 실행 전에 실행
    @Before("execution(* com.tomatoma.service.*.*(..))")
    public void beforeMethod(JoinPoint joinPoint) {
        System.out.println("메서드 실행 전");
    }

    // After: 메서드 실행 후에 실행 (예외 발생 여부 상관없음)
    @After("execution(* com.tomatoma.service.*.*(..))")
    public void afterMethod(JoinPoint joinPoint) {
        System.out.println("메서드 실행 후");
    }

    // AfterReturning: 메서드가 정상 종료 후에 실행
    @AfterReturning(pointcut = "execution(* com.tomatoma.service.*.*(..))",
                    returning = "result")
    public void afterReturningMethod(JoinPoint joinPoint, Object result) {
        System.out.println("메서드 반환값: " + result);
    }

    // AfterThrowing: 메서드에서 예외 발생 시 실행
    @AfterThrowing(pointcut = "execution(* com.tomatoma.service.*.*(..))",
                   throwing = "ex")
    public void afterThrowingMethod(JoinPoint joinPoint, Exception ex) {
        System.err.println("메서드 예외: " + ex.getMessage());
    }

    // Around: 메서드 전후 모두 처리 (가장 강력함)
    @Around("execution(* com.tomatoma.service.*.*(..))")
    public Object aroundMethod(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("메서드 실행 전");
        Object result = joinPoint.proceed();  // 실제 메서드 실행
        System.out.println("메서드 실행 후");
        return result;
    }
}
```

4. JoinPoint (조인포인트)

```
Aspect가 적용될 수 있는 모든 위치 (메서드 호출, 필드 접근 등)
```

3.4 이 프로젝트에 AOP 적용하기

Step 1: pom.xml에 의존성 추가

```
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2: TomatomaApplication.java에 @EnableAspectJAutoProxy 추가

```java
package com.tomatoma;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.EnableAspectJAutoProxy;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
@EnableAspectJAutoProxy  // AOP 활성화
public class TomatomaApplication {

    public static void main(String[] args) {
        SpringApplication.run(TomatomaApplication.class, args);
    }

}
```

Step 3: Aspect 클래스 생성

LoggingAspect.java - Service 계층의 모든 메서드 실행 시간 측정

```java
package com.tomatoma.aspect;

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    private static final Logger logger = LoggerFactory.getLogger(LoggingAspect.class);

    /**
     * Service 계층의 모든 public 메서드 실행을 로깅
     *
     * @param joinPoint 메서드 실행 정보
     * @return 메서드의 반환값
     * @throws Throwable 메서드에서 발생한 예외
     */
    @Around("execution(* com.tomatoma.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();
        String methodName = joinPoint.getSignature().getName();
        String className = joinPoint.getTarget().getClass().getSimpleName();

        logger.info("[START] {}.{}", className, methodName);

        try {
            // 실제 메서드 실행
            Object result = joinPoint.proceed();
```

```
            long duration = System.currentTimeMillis() - startTime;
            logger.info("[END] {}.{} - 실행 시간: {}ms", className, methodName, duration);

            return result;
        } catch (Exception e) {
            long duration = System.currentTimeMillis() - startTime;
            logger.error("[ERROR] {}.{} - 예외 발생 ({}ms): {}",
                        className, methodName, duration, e.getMessage());
            throw e;
        }
    }
}
```

출력 결과:

```
[INFO] [START] TrendService.getAllTrends
[INFO] [END] TrendService.getAllTrends - 실행 시간: 45ms

[INFO] [START] PlacesService.findPlacesByTrendId
[INFO] [END] PlacesService.findPlacesByTrendId - 실행 시간: 28ms

[ERROR] [ERROR] CategoryService.getCategoryById - 예외 발생 (5ms):
        Category not found
```

ExceptionAspect.java - 예외 발생 시 자동으로 로깅

```
package com.tomatoma.aspect;

import org.aspectj.lang.annotation.AfterThrowing;
import org.aspectj.lang.annotation.Aspect;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class ExceptionAspect {

    private static final Logger logger = LoggerFactory.getLogger(ExceptionAspect.class);

    /**
     * Service 계층에서 예외 발생 시 로깅
     *
     * @param ex 발생한 예외
     */
    @AfterThrowing(pointcut = "execution(* com.tomatoma.service.*.*(..))",
                   throwing = "ex")
    public void handleException(Exception ex) {
        logger.error("Service에서 예외 발생: {}", ex.getMessage(), ex);
    }

    /**
     * Repository 계층에서 데이터베이스 예외 발생 시 로깅
     *
     * @param ex 발생한 예외
     */
    @AfterThrowing(pointcut = "execution(* com.tomatoma.repository.*.*(..))",
                   throwing = "ex")
    public void handleRepositoryException(Exception ex) {
        logger.error("데이터베이스 접근 중 예외 발생: {}", ex.getMessage(), ex);
```

```
    }
}
```

3.5 AOP 활용 사례

이 프로젝트에서 AOP를 적용할 수 있는 상황들:

기능	용도	포인트컷
API 로깅	어떤 엔드포인트가 호출되었고, 얼마나 걸렸는가	`@Around` + Controller 계층
성능 모니터링	느린 쿼리 감지	`@Around` + Repository 계층
예외 처리	모든 예외 통일 로깅	`@AfterThrowing` + Service/Repository
보안	API 인증/권한 확인	`@Before` + Controller 계층
트랜잭션	자동 커밋/롤백	`@Around` + Service 계층
캐싱	자주 조회되는 데이터 메모리에 저장	`@Around` + 특정 메서드

---

4. 현재 코드 해석

4.1 CategoryRepository 상세 분석

```
@Repository
public interface CategoryRepository extends JpaRepository<Category, Long> {
    Optional<Category> findByName(String name);
}
```

라인별 해설:

라인	코드	의미
1	`@Repository`	Spring이 이 인터페이스를 데이터베이스 접근 객체로 관리 + 데이터베이스 예외 변환
2	`public interface CategoryRepository`	이것이 인터페이스임을 명시 (구현체는 Spring Data JPA가 자동 생성)
2	`extends JpaRepository<Category, Long>`	JPA의 모든 기본 메서드 상속 (findAll, findById, save, delete 등)
2	`<Category, Long>`	관리하는 Entity는 Category, 주 키 타입은 Long
3	`Optional<Category> findByName(String name);`	개발자 정의 쿼리 메서드 (Spring Data JPA가 자동 구현)

메서드별 동작:

```
// 자동으로 제공되는 메서드들 (JpaRepository 상속)
List<Category> findAll();                    // 모든 카테고리 조회
Optional<Category> findById(Long id);        // 특정 ID의 카테고리 조회
Category save(Category category);             // 카테고리 저장 (없으면 insert, 있으면 update)
void delete(Category category);              // 카테고리 삭제
void deleteById(Long id);                    // 특정 ID의 카테고리 삭제
long count();                                // 카테고리 개수 세기
boolean existsById(Long id);                 // 특정 ID 존재 여부 확인

// 개발자 정의 메서드
Optional<Category> findByName(String name);  // 이름으로 카테고리 조회
// 자동 생성 SQL: SELECT * FROM categories WHERE name = ?
```

4.2 data.sql과 테이블 스키마 매핑

Category Entity:

```
@Entity
@Table(name = "categories")
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @Column(length = 7)
    private String color;

    @Column(length = 10)
    private String icon_emoji;
}
```

생성되는 테이블 스키마:

```
CREATE TABLE categories (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,
    color VARCHAR(7),
    icon_emoji VARCHAR(10)
);
```

data.sql의 INSERT 문:

```
INSERT INTO categories (name, color, icon_emoji) VALUES ('한식', '#FF5252', '🍱');
```

매핑 과정:

```
INSERT 문 실행
    ↓
categories 테이블의 새 행 추가
    ↓
JPA가 자동으로 Entity 객체로 변환
    ↓
CategoryRepository.findByName("한식") 사용 가능
```

4.3 TrendFood, FoodPlace Entity 분석

TrendFood (트렌드 음식):

```
@Entity
@Table(name = "trend_foods")
public class TrendFood {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;                // 음식 이름: "버터 계란 밥"

    @Column(length = 50)
    private String category;            // 카테고리: "한식"
```

```java
    @Column
    private Integer search_frequency;     // 검색 빈도: 8500

    @Column
    private Integer trend_rank;           // 순위: 1

    @Column(length = 500)
    private String image_url;             // 이미지 URL

    @Column(length = 50)
    private String source;                // 출처: "google_trends"

    @Column(nullable = false, updatable = false)
    private LocalDateTime created_at;     // 생성 시간 (수정 불가)

    @Column(nullable = false)
    private LocalDateTime updated_at;     // 수정 시간
}
```

FoodPlace (음식점):

```java
@Entity
@Table(name = "food_places")
public class FoodPlace {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private Long trend_food_id;           // 어떤 트렌드 음식의 위치인가 (외래키)

    @Column(nullable = false, length = 200)
    private String name;                  // 음식점 이름: "명동 계란밥"

    @Column(nullable = false, length = 300)
    private String address;               // 주소: "서울시 중구 명동 111"

    @Column(nullable = false)
    private Double latitude;              // 위도: 37.5642

    @Column(nullable = false)
    private Double longitude;             // 경도: 126.9847

    @Column(length = 20)
    private String phone;                 // 전화: "02-1234-5678"

    @Column
    private Double rating;                // 별점: 4.5

    @Column
    private Integer price_approx;         // 예상 가격: 7500 (원)

    @Column(length = 200)
    private String operating_hours;       // 영업 시간: "10:00 - 22:00"

    @Column(length = 100)
    private String google_place_id;       // Google Places ID (외부 API용)

    @Column(nullable = false, updatable = false)
    private LocalDateTime created_at;
```

```
    @Column(nullable = false)
    private LocalDateTime updated_at;
}
```

관계:

```
TrendFood(트렌드 음식) ──────────1:N──────────→ FoodPlace(음식점)

예:
- TrendFood ID=1 (버터 계란 밥)
  ├── FoodPlace ID=1 (명동 계란밥, trend_food_id=1)
  └── FoodPlace ID=2 (강남 버터 밥, trend_food_id=1)

- TrendFood ID=4 (카라멜 마끼아또)
  └── FoodPlace ID=3 (역삼 카페, trend_food_id=4)
```

---

5. REST API 엔드포인트 설계

5.1 설계된 API 목록

현재 프로젝트에서 구현할 예정인 API들:

HTTP 메서드	엔드포인트	설명	반환 데이터
GET	/api/trends	모든 트렌드 음식 조회	List
GET	/api/trends/{id}	특정 트렌드 음식 조회	TrendFoodDTO
GET	/api/trends/search?q=버터	트렌드 음식 검색	List
POST	/api/trends	새 트렌드 음식 추가	TrendFoodDTO
PUT	/api/trends/{id}	트렌드 음식 수정	TrendFoodDTO
DELETE	/api/trends/{id}	트렌드 음식 삭제	-
GET	/api/places/{trendFoodId}	특정 음식의 위치 조회	List
GET	/api/places	모든 음식점 조회	List
POST	/api/places	새 음식점 추가	FoodPlaceDTO
PUT	/api/places/{id}	음식점 정보 수정	FoodPlaceDTO
DELETE	/api/places/{id}	음식점 삭제	-
GET	/api/categories	모든 카테고리 조회	List
POST	/api/categories	새 카테고리 추가	CategoryDTO

5.2 각 API의 요청/응답 예시

1. 모든 트렌드 음식 조회

```
GET /api/trends HTTP/1.1
Host: localhost:8080
```

응답 (200 OK):

```
{
  "status": "success",
  "data": [
    {
```

```
      "id": 1,
      "name": "버터 계란 밥",
      "category": "한식",
      "searchFrequency": 8500,
      "trendRank": 1,
      "imageUrl": "https://example.com/image1.jpg",
      "source": "google_trends"
    },
    {
      "id": 2,
      "name": "계란 마니아",
      "category": "한식",
      "searchFrequency": 7200,
      "trendRank": 2,
      "imageUrl": "https://example.com/image2.jpg",
      "source": "google_trends"
    }
  ],
  "error": null
}
```


2. 특정 음식의 위치 조회

```
GET /api/places/1 HTTP/1.1
Host: localhost:8080
```

응답 (200 OK):


```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "trendFoodId": 1,
      "name": "명동 계란밥",
      "address": "서울시 중구 명동 111",
      "latitude": 37.5642,
      "longitude": 126.9847,
      "phone": "02-1234-5678",
      "rating": 4.5,
      "priceApprox": 7500,
      "operatingHours": "10:00 - 22:00",
      "googlePlaceId": "place_id_001"
    },
    {
      "id": 2,
      "trendFoodId": 1,
      "name": "강남 버터 밥",
      "address": "서울시 강남구 강남대로 222",
      "latitude": 37.4979,
      "longitude": 127.0276,
      "phone": "02-2222-3333",
      "rating": 4.3,
      "priceApprox": 8000,
      "operatingHours": "11:00 - 23:00",
      "googlePlaceId": "place_id_002"
    }
  ],
  "error": null
}
```

3. 모든 카테고리 조회

```
GET /api/categories HTTP/1.1
Host: localhost:8080
```

응답 (200 OK):


```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "한식",
      "color": "#FF5252",
      "iconEmoji": "🍚"
    },
    {
      "id": 2,
      "name": "양식",
      "color": "#AB47BC",
      "iconEmoji": "🍝"
    },
    {
      "id": 3,
      "name": "중식",
      "color": "#FF9800",
      "iconEmoji": "🥡"
    }
  ],
  "error": null
}
```


4. 새 트렌드 음식 추가

```
POST /api/trends HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{
  "name": "치즈 떡볶이",
  "category": "분식",
  "searchFrequency": 6000,
  "trendRank": 10,
  "imageUrl": "https://example.com/cheese-tteokbokki.jpg",
  "source": "naver_trends"
}
```

응답 (201 Created):


```
{
  "status": "success",
  "data": {
    "id": 6,
    "name": "치즈 떡볶이",
    "category": "분식",
    "searchFrequency": 6000,
    "trendRank": 10,
    "imageUrl": "https://example.com/cheese-tteokbokki.jpg",
    "source": "naver_trends",
    "createdAt": "2025-01-15T14:30:00",
    "updatedAt": "2025-01-15T14:30:00"
```

```
  },
  "error": null
}
```

5. 트렌드 음식 검색

```
GET /api/trends/search?q=계란 HTTP/1.1
Host: localhost:8080
```

응답 (200 OK):

```
{
  "status": "success",
  "data": [
    {
      "id": 1,
      "name": "버터 계란 밥",
      "category": "한식",
      "searchFrequency": 8500,
      "trendRank": 1
    },
    {
      "id": 2,
      "name": "계란 마니아",
      "category": "한식",
      "searchFrequency": 7200,
      "trendRank": 2
    }
  ],
  "error": null
}
```

5.3 API 계층 흐름 정리

GET /api/trends

```
TrendFoodController
    ↓
    @GetMapping
    public ResponseEntity<ResponseDTO> getAllTrends()

    ↓

TrendService
    ↓
    public List<TrendFoodDTO> getAllTrends()

    ↓

TrendFoodRepository
    ↓
    findAll() 호출

    ↓

데이터베이스
    ↓
    SELECT * FROM trend_foods

    ↓
```

```
결과 반환 (역순)
    ↓
TrendFoodRepository: List<TrendFood> 반환
    ↓
TrendService: Entity → DTO 변환, List<TrendFoodDTO> 반환
    ↓
TrendFoodController: ResponseDTO에 담기, JSON 변환
    ↓
클라이언트: JSON 응답 수신
```

GET /api/places/{trendFoodId}

```
FoodPlaceController
    ↓
    @GetMapping("/{trendFoodId}")
    public ResponseEntity<ResponseDTO> getPlacesByTrendFood(@PathVariable Long trendFoodId)

    ↓

PlacesService
    ↓
    public List<FoodPlaceDTO> findPlacesByTrendId(Long trendFoodId)

    ↓

FoodPlaceRepository
    ↓
    findByTrendFoodId(trendFoodId) 호출

    ↓

데이터베이스
    ↓
    SELECT * FROM food_places WHERE trend_food_id = ?

    ↓

결과 반환 (역순)
```

---

6. 구현할 것 (TODO 체크리스트)

6.1 아직 구현되지 않은 파일 목록

파일 경로	역할	우선순위
`entity/TrendFood.java`	트렌드 음식 엔티티	1단계 ✓ (완료)
`entity/FoodPlace.java`	음식점 엔티티	1단계 ✓ (완료)
`entity/Category.java`	카테고리 엔티티	1단계 ✓ (완료)
`repository/TrendFoodRepository.java`	트렌드 음식 Repository	2단계
`repository/FoodPlaceRepository.java`	음식점 Repository	2단계
`repository/CategoryRepository.java`	카테고리 Repository	2단계 ✓ (완료)
`dto/TrendFoodDTO.java`	트렌드 음식 DTO	3단계
`dto/FoodPlaceDTO.java`	음식점 DTO	3단계
`dto/CategoryDTO.java`	카테고리 DTO	3단계

dto/ResponseDTO.java	공통 응답 DTO	3단계
service/TrendService.java	트렌드 음식 서비스	4단계
service/PlacesService.java	음식점 서비스	4단계
service/CategoryService.java	카테고리 서비스	4단계
controller/TrendFoodController.java	트렌드 음식 컨트롤러	5단계
controller/FoodPlaceController.java	음식점 컨트롤러	5단계
controller/CategoryController.java	카테고리 컨트롤러	5단계
scheduler/TrendUpdaterScheduler.java	트렌드 자동 업데이트	6단계
exception/GlobalExceptionHandler.java	전역 예외 처리	6단계
config/CorsConfig.java	CORS 설정	7단계
aspect/LoggingAspect.java	로깅 AOP	8단계
aspect/ExceptionAspect.java	예외 처리 AOP	8단계

6.2 구현 순서 및 설명

1단계: Entity 클래스 (데이터 구조 정의)

목표: 데이터베이스 테이블의 Java 표현

파일:

- `Category.java` ✓ (완료)
- `TrendFood.java` ✓ (완료)
- `FoodPlace.java` ✓ (완료)

포함 내용:

- `@Entity`, `@Table`, `@Id`, `@Column` 어노테이션
- Getter, Setter
- equals, hashCode, toString

2단계: Repository 인터페이스 (데이터베이스 접근)

목표: 데이터베이스와의 통신 정의

파일:

- `CategoryRepository.java` ✓ (완료)
- `TrendFoodRepository.java` (미완료)
- `FoodPlaceRepository.java` (미완료)

포함 내용:

```
@Repository
public interface TrendFoodRepository extends JpaRepository<TrendFood, Long> {
    List<TrendFood> findByCategory(String category);
    Optional<TrendFood> findByName(String name);
    List<TrendFood> findByNameContaining(String keyword);  // 검색용
    List<TrendFood> findAllOrderByTrendRankAsc();          // 순위 정렬
}
```

3단계: DTO 클래스 (통신용 데이터)

목표: 클라이언트에 보낼 데이터 구조 정의

파일:

- `TrendFoodDTO.java` (미완료)

- `FoodPlaceDTO.java` (미완료)
- `CategoryDTO.java` (미완료)
- `ResponseDTO.java` (미완료)

예시:

```
public class TrendFoodDTO {
    private Long id;
    private String name;
    private String category;
    private Integer searchFrequency;
    private Integer trendRank;
    private String imageUrl;
    private String source;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    // Getter, Setter, 생성자 등
}

public class ResponseDTO<T> {
    private String status;      // "success" or "error"
    private T data;             // 실제 데이터
    private String error;       // 에러 메시지

    public ResponseDTO(String status, T data, String error) {
        this.status = status;
        this.data = data;
        this.error = error;
    }
}
```

4단계: Service 클래스 (비즈니스 로직)

목표: 데이터 처리 및 변환 로직

파일:

- `TrendService.java` (미완료)
- `PlacesService.java` (미완료)
- `CategoryService.java` (미완료)

예시:

```
@Service
public class TrendService {

    private final TrendFoodRepository trendFoodRepository;

    public TrendService(TrendFoodRepository trendFoodRepository) {
        this.trendFoodRepository = trendFoodRepository;
    }

    // 모든 트렌드 조회 (Entity → DTO 변환)
    public List<TrendFoodDTO> getAllTrends() {
        return trendFoodRepository.findAll()
            .stream()
            .map(this::toDTO)
            .collect(Collectors.toList());
    }

    // 트렌드 이름으로 검색
    public List<TrendFoodDTO> searchByName(String keyword) {
```

```
        return trendFoodRepository.findByNameContaining(keyword)
            .stream()
            .map(this::toDTO)
            .collect(Collectors.toList());
    }

    // Entity를 DTO로 변환
    private TrendFoodDTO toDTO(TrendFood entity) {
        TrendFoodDTO dto = new TrendFoodDTO();
        dto.setId(entity.getId());
        dto.setName(entity.getName());
        dto.setCategory(entity.getCategory());
        // ... 나머지 필드도 매핑
        return dto;
    }
}
```

5단계: Controller 클래스 (HTTP 요청 처리)

목표: REST API 엔드포인트 정의

파일:

- `TrendFoodController.java` (미완료)
- `FoodPlaceController.java` (미완료)
- `CategoryController.java` (미완료)

예시:

```
@RestController
@RequestMapping("/api/trends")
public class TrendFoodController {

    private final TrendService trendService;

    public TrendFoodController(TrendService trendService) {
        this.trendService = trendService;
    }

    @GetMapping
    public ResponseEntity<ResponseDTO<List<TrendFoodDTO>>> getAllTrends() {
        List<TrendFoodDTO> trends = trendService.getAllTrends();
        ResponseDTO<List<TrendFoodDTO>> response =
            new ResponseDTO<>("success", trends, null);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/search")
    public ResponseEntity<ResponseDTO<List<TrendFoodDTO>>> searchTrends(
        @RequestParam String q
    ) {
        List<TrendFoodDTO> results = trendService.searchByName(q);
        ResponseDTO<List<TrendFoodDTO>> response =
            new ResponseDTO<>("success", results, null);
        return ResponseEntity.ok(response);
    }
}
```

6단계: Scheduler & Exception Handler

목표: 자동 업데이트 및 통일된 예외 처리

파일:

- `TrendUpdaterScheduler.java` (미완료)
- `GlobalExceptionHandler.java` (미완료)

TrendUpdaterScheduler.java:

```java
@Service
public class TrendUpdaterScheduler {

    private final TrendService trendService;
    private static final Logger logger = LoggerFactory.getLogger(TrendUpdaterScheduler.class);

    public TrendUpdaterScheduler(TrendService trendService) {
        this.trendService = trendService;
    }

    @Scheduled(cron = "0 0 * * * ?")  // 매시간 정각
    public void updateTrendFoods() {
        logger.info("트렌드 데이터 업데이트 시작");

        try {
            // 외부 API에서 최신 데이터 조회 (구현 필요)
            List<TrendFood> latestTrends = fetchFromExternalAPI();

            // 데이터베이스에 저장
            trendService.saveAllTrends(latestTrends);

            logger.info("트렌드 데이터 업데이트 완료");
        } catch (Exception e) {
            logger.error("트렌드 업데이트 중 에러", e);
        }
    }
}
```

GlobalExceptionHandler.java:

```java
@RestControllerAdvice  // 모든 Controller에서 발생한 예외를 처리
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(EntityNotFoundException.class)
    public ResponseEntity<ResponseDTO<Void>> handleEntityNotFound(EntityNotFoundException ex) {
        logger.warn("데이터를 찾을 수 없음: {}", ex.getMessage());
        ResponseDTO<Void> response = new ResponseDTO<>(
            "error",
            null,
            ex.getMessage()
        );
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(response);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ResponseDTO<Void>> handleGeneralException(Exception ex) {
        logger.error("예외 발생", ex);
        ResponseDTO<Void> response = new ResponseDTO<>(
            "error",
            null,
            "서버 오류가 발생했습니다"
        );
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(response);
```

```
    }
}
```

7단계: Config 설정

목표: CORS, 보안 등 설정

파일:

- `CorsConfig.java` (미완료)
- `SecurityConfig.java` (미완료)

8단계: AOP (선택사항)

목표: 로깅, 성능 모니터링 등 횡단 관심사 처리

파일:

- `LoggingAspect.java` (미완료)
- `ExceptionAspect.java` (미완료)

---

7. 실행 및 테스트

7.1 프로젝트 실행

```
# Maven 빌드
mvn clean install

# Spring Boot 실행
mvn spring-boot:run

# 또는 IDE에서 TomatomaApplication.java 우클릭 → Run
```

7.2 데이터베이스 확인

```
http://localhost:8080/h2-console

JDBC URL: jdbc:h2:mem:tomatoma
User Name: sa
Password: (비워두기)
```

7.3 API 테스트 (curl 또는 Postman)

```
# 모든 카테고리 조회
curl http://localhost:8080/api/categories

# 특정 음식점 조회
curl http://localhost:8080/api/places/1

# 검색
curl "http://localhost:8080/api/trends/search?q=계란"
```

---

8. 학습 팁

8.1 개념 이해 순서

1. **Entity** 이해하기
    - 클래스 = 테이블, 필드 = 컬럼
    - `@Id`, `@Column` 어노테이션의 역할

2. **Repository** 이해하기
    - JpaRepository 상속만으로 기본 CRUD 가능
    - 쿼리 메서드의 자동 생성 원리
3. **DTO** 이해하기
    - Entity와 다른 이유 (보안, 선택적 필드)
    - Entity → DTO 변환 방법
4. **Service** 이해하기
    - 비즈니스 로직의 중심
    - Repository와 Controller 사이의 중간자 역할
5. **Controller** 이해하기
    - HTTP 메서드와 경로의 매핑
    - 요청/응답의 변환
6. **전체 흐름** 이해하기
    - Request → Controller → Service → Repository → DB → (역순) Response

8.2 디버깅 팁

```yaml
# application.yml에서 로그 레벨 조정
logging:
  level:
    com.tomatoma: DEBUG          # 우리 코드 DEBUG로
    org.springframework.web: DEBUG  # 웹 프레임워크도 DEBUG
    org.hibernate.SQL: DEBUG     # SQL 쿼리 출력
    org.hibernate.type: TRACE    # SQL 파라미터 출력
```

8.3 참고 자료

- Spring Boot 공식 문서
- Spring Data JPA 쿼리 메서드
- Spring AOP
- REST API 설계 가이드

최종 정리

핵심 원리

```
데이터 흐름:
Client (JSON)
    ↓
Controller (@RestController가 자동 변환)
    ↓
Service (비즈니스 로직)
    ↓
Repository (JPA가 자동 SQL 생성)
    ↓
Database (H2/PostgreSQL)
    ↓
(역순)
    ↓
Client (JSON)
```

각 계층의 책임

계층	책임	도구

Controller	HTTP 요청 받고 응답 보내기	@RestController, @GetMapping 등
Service	비즈니스 로직, 데이터 변환	@Service, 메인 로직
Repository	데이터베이스 접근	@Repository, JpaRepository
Entity	데이터베이스 구조 표현	@Entity, @Column 등
DTO	클라이언트와 통신용	보안, 성능 최적화

구현 우선순위

1. Entity 정의 (데이터 구조)
2. Repository 정의 (DB 접근)
3. DTO 정의 (통신 구조)
4. Service 구현 (비즈니스 로직)
5. Controller 구현 (HTTP 엔드포인트)
6. 예외 처리 및 로깅 (운영 안정성)
7. AOP, 성능 최적화 (심화)

문서 작성 완료

이 학습 자료는 Tomatoma Spring Boot 백엔드 프로젝트의 완전 초보자를 위한 가이드입니다. 각 섹션을 순서대로 이해하면, 백엔드 웹 개발의 기초를 탄탄히 다질 수 있습니다.